CS6843 Program Analysis
Assignment 2 : Range Analysis

Deadline: September 7, 2025 Marks: 15

1 Problem Statement

The objective of this assignment is to implement a *Flow-sensitive Intra-procedural Range analysis* for local integer variables of a function. The analysis must compute the possible range of values (i.e. [lower bound, upper bound] both inclusive) that a local integer variable can have, at the end of the function.

1.1 Input

In order to simplify the assignment, the input program will have the following restrictions.

- The program will only have local variables of type `int` (32-Bit Integers), and all of them will be declared at the beginning of the function. The function arguments will also be of the same type.
- There will not be any global variables, static variables, pointers, structures, unions or arrays in the program. The analysis can focus only on the integer local variables.
- The program will only have addition, subtraction and multiplication operations on variables. The analysis can ignore other arithmetic operations.
- Conditions in loops and If-else control structures, will be of the form : `(var op k)` , where `var` will be a local variable, `op` represents one of $< \leq > \geq$ comparison operators and `k` is an integer constant.
- There will not be any nested loops in the program. (Every other control structure except nested loops have to be handled.)
- Each function will have a maximum of 15 lines of code.

1.2 Output

For every function in the program, print the function name as

$$Function < function_name >$$

Followed by the ranges (at the end of the function) computed for each of the local variables and arguments (one per line in any order) as

$$< variable_name >: [< min_value >:< max_value >]$$

For example if a variable "var" can take values from 0 to 10 in function main, the output should be

```
Function main
var : [0,10]
```

2 Examples

Example 1:

```
1  void test (int arg) {
2      int i = 20, j = 30;
3      // i : [20,20], j : [30,30]
4      if (arg > 0) {
5          i += 40;
6          // i : [60,60], j : [30,30]
7      }
8      // i : [20,60], j : [30,30]
9      j = i + j;
10     // i : [20,60], j : [50,90]
11 }
12
13 int main () {
14     test(10);
15     return 0;
16 }
```

Expected Output:

```
1 Function test
2 arg : [-2147483648,2147483647]
3 i : [20,60]
4 j : [50,90]
5
6 Function main
```

Explanation:

In function `test`, the input argument `arg` has the full 32-bit integer range (from `INT_MIN` to `INT_MAX`), since it is a function argument and was never overwritten inside the function. The variable `i` gets the value `20` at line 2, and may get updated to 60 in line 4, so the range of `i` at the end of the function is `[20, 60]`. The variable `j` gets the value `30` at line 2. At line 6, `j` gets incremented with the value of variable `i` which at line 6 has the range `[20,60]`, and so, the final range of `j` is `[50,90]`.

Function `main` doesn't have any variable or arguments and so we print nothing.

Example 2:

```
1  int main () {
2      int a = 5, n;
3      scanf("%d", &n);
4      // a : [5,5], n : [-2147483648,2147483647]
5      if (n >= 3) {
6          // a : [5,5], n : [3,2147483647]
7          a = n;
8          // a : [3,2147483647], n : [3,2147483647]
9      }
10     // a : [3,2147483647], n : [-2147483648,2147483647]
11     return 0;
12 }
```

Expected Output:

```
1 Function main
2 n : [-2147483648,2147483647]
3 a : [3,2147483647]
```

Explanation:

In the `main` function, because of the `if` statement's condition, in the true branch, the range of `n` is from `3` to `INT_MAX`, which gets assigned to `a`. The original range of `a` (`[5,5]`) gets subsumed into the `[3,INT_MAX]` range, at the end of the function.

Example 3:

```c
int main () {
    int i = 0, j = 10;
    // i : [0,0], j : [10,10]
    for (i = 0; i < 10; i++) {
        j += 1;      // j : [11,11]
        printf("%d\n", j);
        j -= 1;      // j : [10,10]
        printf("%d\n", j);
    }
    // i : [10,2147483647], j : [10,10]
    return 0;
}
```

Expected Output:

```
Function main
i : [-2147483648,2147483647]
j : [10,10]
```

Explanation:

In the `main` function, for any iteration of the `for` loop, the range of `j` at the end of the iteration remains the same as the beginning of the iteration, and so it retains the range. However, since the value of `i` changes during the iteration of the loop, the analysis conservatively sets it to `[INT_MIN,INT_MAX]` at the head of the loop. Finally, due to the loop condition, only values of `i` greater than or equal to `10` can go out of the loop, and so, the range of `i` at the end of the function is `[10,INT_MAX]`.

3 Template

The provided template contains the following files, folders and helper scripts:

- `analysis/rangeAnalysis.cc` : The file containing the LLVM function pass template. You are expected to implement all your analysis logic in this file.
- `testcases/` : The testcases directory contains the public testcases. Each testcase consists of a C file, the corresponding LLVM IR file and the expected output from the analysis.
- `build.sh` : Helper script that compiles the analysis pass.
- `run.sh <inputfile.c/inputfile.ll>` : Helper script that compiles and runs the analysis on the given input file (in C or in LLVM IR format (.ll)).
- `run_testcases.sh` : Compiles the analysis and runs all the public testcases on the analysis. This script uses *lit* (LLVM Integrated Tester) for testing, and will install *lit* on the first run.

4 Analysis details

Implementing Range class and Data flow analysis : `rangeAnalysis.cc` partially implements `Range` class to represent an integer range. You are expected to complete the implementation, and use this class to represent the data flow fact in the analysis. You are also expected to implement the Kildall's algorithm to perform data flow analysis. You are not allowed to use any in-built LLVM classes for representing a range, or for performing data flow analysis.
Handling integer overflow/underflow : You can assume that there are no overflows or underflows caused due to arithmetic on integers. However, in the analysis, you may have to perform arithmetic on `INT_MAX` and `INT_MIN` values (or values close to them). In such cases, follow *Saturating semantics*, i.e. if the result of an arithmetic operation is greater than `INT_MAX`,

set it to `INT_MAX`. Similarly, if the result of an arithmetic operation is smaller than `INT_MIN`, set it to `INT_MIN`.

Handling conditional branches : As illustrated in example 2, the analysis needs to be aware of conditional branches. For instance, if the condition in an `if` statement is `(n >= 3)`, then at the beginning of the true branch, the range of `n` cannot have any values less than 3. Similarly, at the beginning of the false (else) branch, the range of `n` cannot have any values greater than or equal to 3. Implementing this requires special handling when performing a meet of values from the predecessor basic blocks.

Handling loops : As illustrated in example 3, the analysis is conservative in handling loops. If the range of any variable (including the iteration variable) changes during the iteration of the loop, the analysis sets it's range to `[INT_MIN,INT_MAX]` at the head of the loop. Implementing this requires handling *back edges* as a special case. Back edge refers to the edges of the control flow graph that create loops. To get a list of all back edges in the function, you can use the in-built LLVM function `llvm::FindFunctionBackedges(&F,...)`.

5 Submission Guidelines

- Submit only the `analysis/rangeAnalysis.cc` file renamed as `ROLLNO.cc` on Moodle. Ensure that all your code is implemented in the same file.
- Evaluation will be based on test cases. Ensure that your code prints the results in the expected format and passes the public testcases provided.